

Программирование на Python, БИ

НИУ ВШЭ, 2025–26 учебный год

✓ Семинар 6. Visualizations

✓ Раздел 1. Основы Matplotlib

Начнём наш разговор, пожалуй, с того, что, вообще говоря, вы, конечно, можете строить графики и использовать для применения визуализаций в вашей работе — а не в рамках нашего курса — любые программные средства и любые модули, которые вам удобны, — не обязательно Matplotlib, да и в принципе не обязательно библиотеки Python. В то же время мы со своей стороны на дисциплине предлагаем вам к изучению — и даже тратим на это довольно много времени — именно этот непростой инструмент. Почему же всё-таки так?

Библиотека Matplotlib — а вместе с тем и все расширяющие, и дополняющие её последующие библиотеки типа Seaborn — хотя и могут на первый взгляд казаться существенно сложнее того же хорошо знакомого нам Excel, — в то же время предоставляют гораздо большую гибкость; изложение становится более плавным, последовательным — например, как в этом ноутбуке, текст подкрепляется графиками и кодом; — а также появляется возможность визуализировать наборы данных такого объема, с которыми Excel в принципе не может работать. Последнее, возможно, является одним из наиболее критичных аргументов, особенно для ряда специализированных областей.

Так что давайте не будем тратить время: импортируем нашу библиотеку, пропишем магическую функцию, позволяющую отображать графики прямо в самом ноутбуке, и непосредственно приступим с вами к важному разговору про Matplotlib.

```
import matplotlib.pyplot as plt
%matplotlib inline
```

[Куча примеров и best practices](#) по построению графиков.

✓ Функция Plot и основные настройки

Самой базовой и простейшей визуализацией в Matplotlib, которую вы можете быстро и без каких-либо сложностей построить, почти ничего не зная про библиотеку, — является обычный линейный Plot. Именно на примере этого графика предлагается начать разбор концептуальных основ Matplotlib'a; а заодно в дополнение мы сможем рассмотреть основные параметры и настройки, которые в дальнейшем будут применяться практически в каждом графике, который можно построить.

Чтобы мы с вами сразу могли обсуждать всё на практике, давайте возьмём знакомую нам из математики функцию синуса и сгенерируем пока что синтетические данные для неё на отрезке от 0 до 3π. Это будет нашим первым тренировочным примером,

разобравшись с которым, уже можно будет постепенно переходить к реальным данным.

```
import numpy as np
```

```
x = np.arange(0, 3 * np.pi, 0.1)  
y = np.sin(x)
```

Что произошло в ячейке выше? Что будет храниться в переменных `x` и `y`?

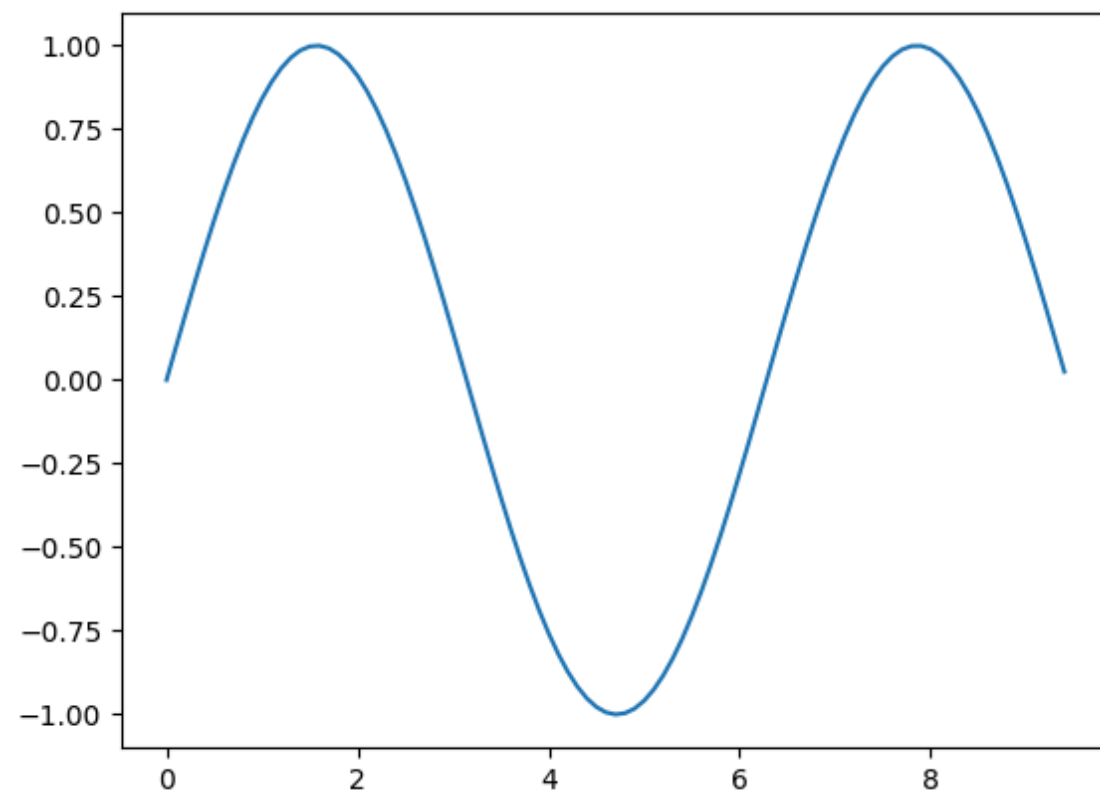
Для того чтобы построить наш первый график, воспользовавшись функцией `plt.plot`, нам необходимо передать ей в качестве параметров минимум 2 аргумента: массив значений для оси `x` и массив значений для оси `y`.

Как и любая функция, осуществляющая визуализацию простого линейного Plot, данная функция вначале наносит на график точки, представляя их парами координат вида: $(x_1; y_1)$, $(x_2; y_2)$, ..., $(x_n; y_n)$, — а затем просто аппроксимирует зависимость по ним с помощью непрерывной гладкой линии. Разумеется, чем больше точек вы укажете в рамках массивов для графика, тем плавнее и красивее получится эта линия, построенная программой.

Кроме того, из всего выше следует, что размерности `x` и `y` должны обязательно совпадать — без этого ничего просто не будет работать.

```
plt.plot(x, y)
```

```
[<matplotlib.lines.Line2D at 0x7b64692f9fd0>]
```



Ура! Мы построили наш первый график. Пока что ничего особо сложного, как вы можете видеть! :)

И он даже похож на то, что мы бы ожидали получить от синуса.. :)

В примере, рассмотренном выше, мы используем так называемый функциональный стиль визуализации в Matplotlib, который очень похож на стиль из Matlab (если вдруг кто-то знает такой пакет, активно использовавшийся ещё несколько лет назад). Идея этого функционального стиля заключается в том, что при построении визуализации мы оперируем глобальным объектом `plt` ("рисовалка") и все команды и действия осуществляем, обращаясь только к нему.

Существует также и альтернативный вариант — так называемый объектный стиль, — когда мы явно задаём и затем оперируем всей "фигурой" и "осями" для нашего графика.

К объектному стилю визуализации мы ещё сегодня вернёмся позднее, а пока что давайте в рамках функционального стиля рассмотрим функцию `plt.figure`. Данная команда в качестве параметров принимает такие свойства, как размер всего графика ("фигуры") — `figsize`; или цвет фона графика — `facecolor`; и множество других. С полным списком доступных параметров для `plt.figure` можно ознакомиться, например, [здесь](#).

Помимо этого в глобальном объекте `plt` доступны и другие методы: например, мы можем выставить графику заголовок с помощью `plt.title`, можем подписать оси с помощью `plt.xlabel` и `plt.ylabel`, а также сделать много чего ещё. Стоит отметить, что всё это оформление считается хорошим тоном при построении графиков.

Итак, давайте продолжим работать с нашим графиком синуса и постепенно будем его модифицировать, применяя к нему новые функции, параметры и используя новые детали оформления. Чтобы все изменения отслеживать было проще, давайте, начиная с ближайшей ячейки и далее, помечать обновленные и/или добавленные нами блоки, а также все прочие изменения — с помощью символов `###`.

Для начала укажем заголовок, нанесем подписи осей, а также настроим размер фигуры для более красивого отображения:

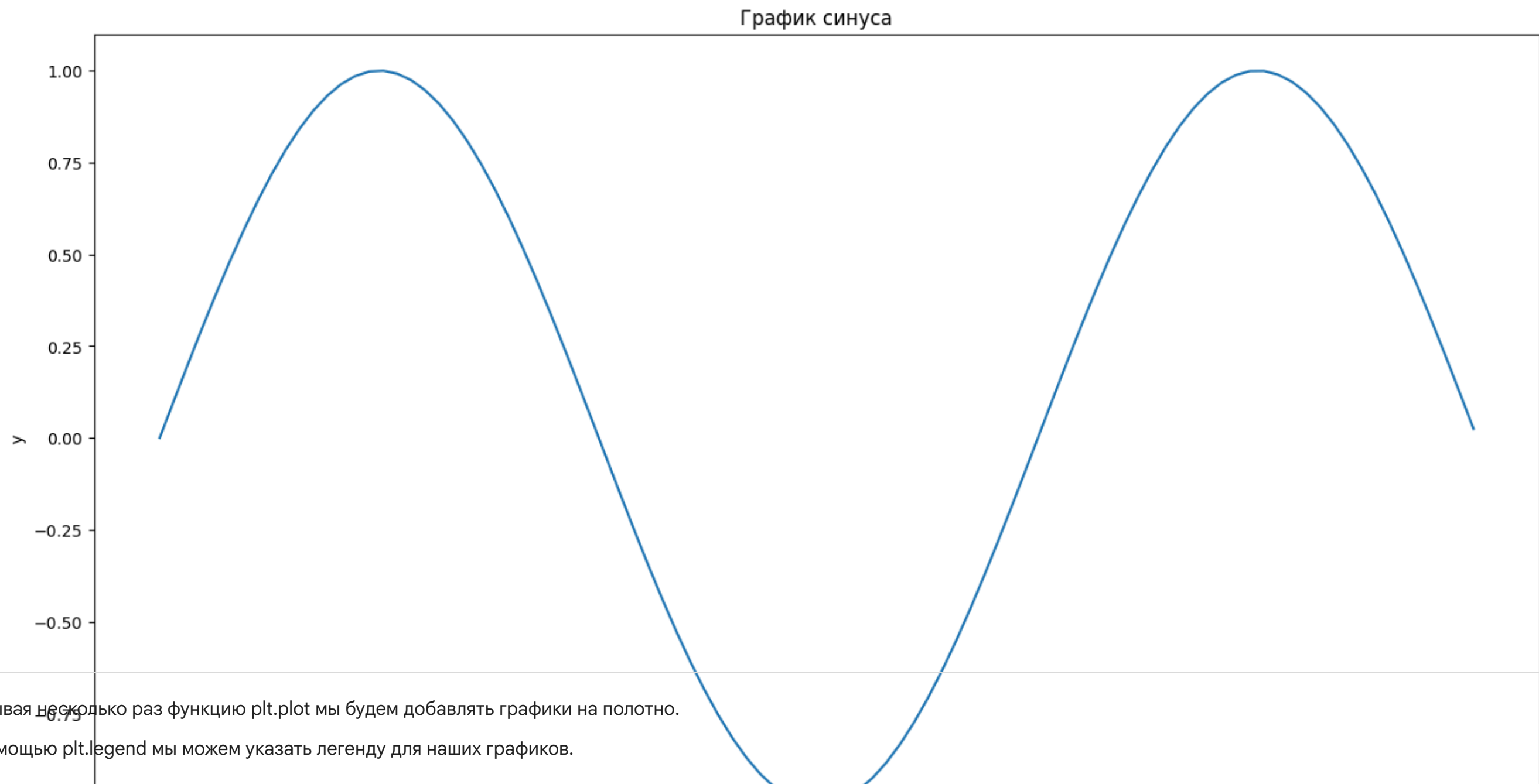
```
x = np.arange(0, 3 * np.pi, 0.1)
y = np.sin(x)

###
plt.figure(figsize=(16,9))
###

plt.plot(x, y)

###
plt.title("График синуса")
plt.xlabel("x")
plt.ylabel("y")

plt.show()
###
```

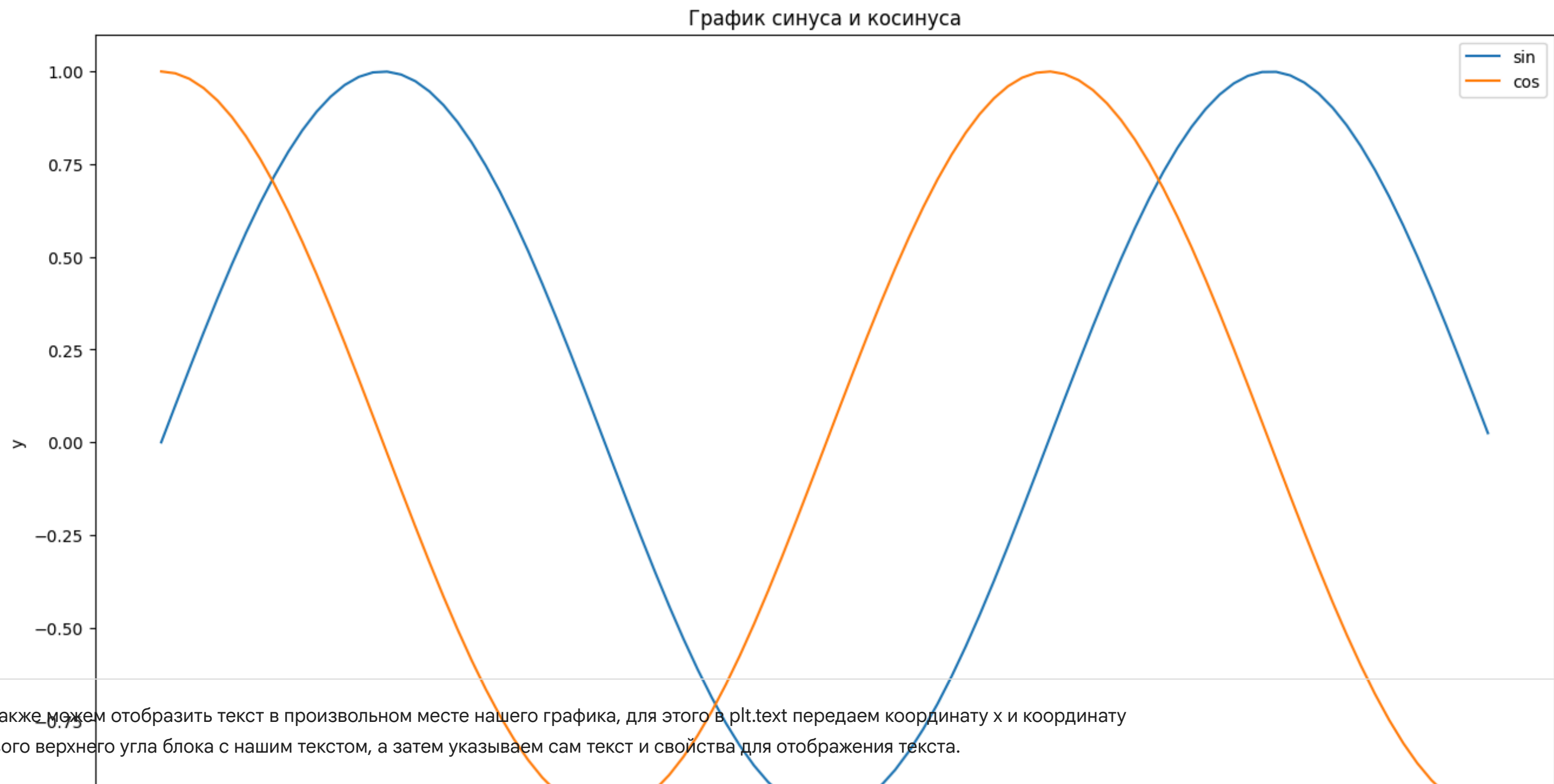


Вызывая несколько раз функцию `plt.plot` мы будем добавлять графики на полотно.

С помощью `plt.legend` мы можем указать легенду для наших графиков.

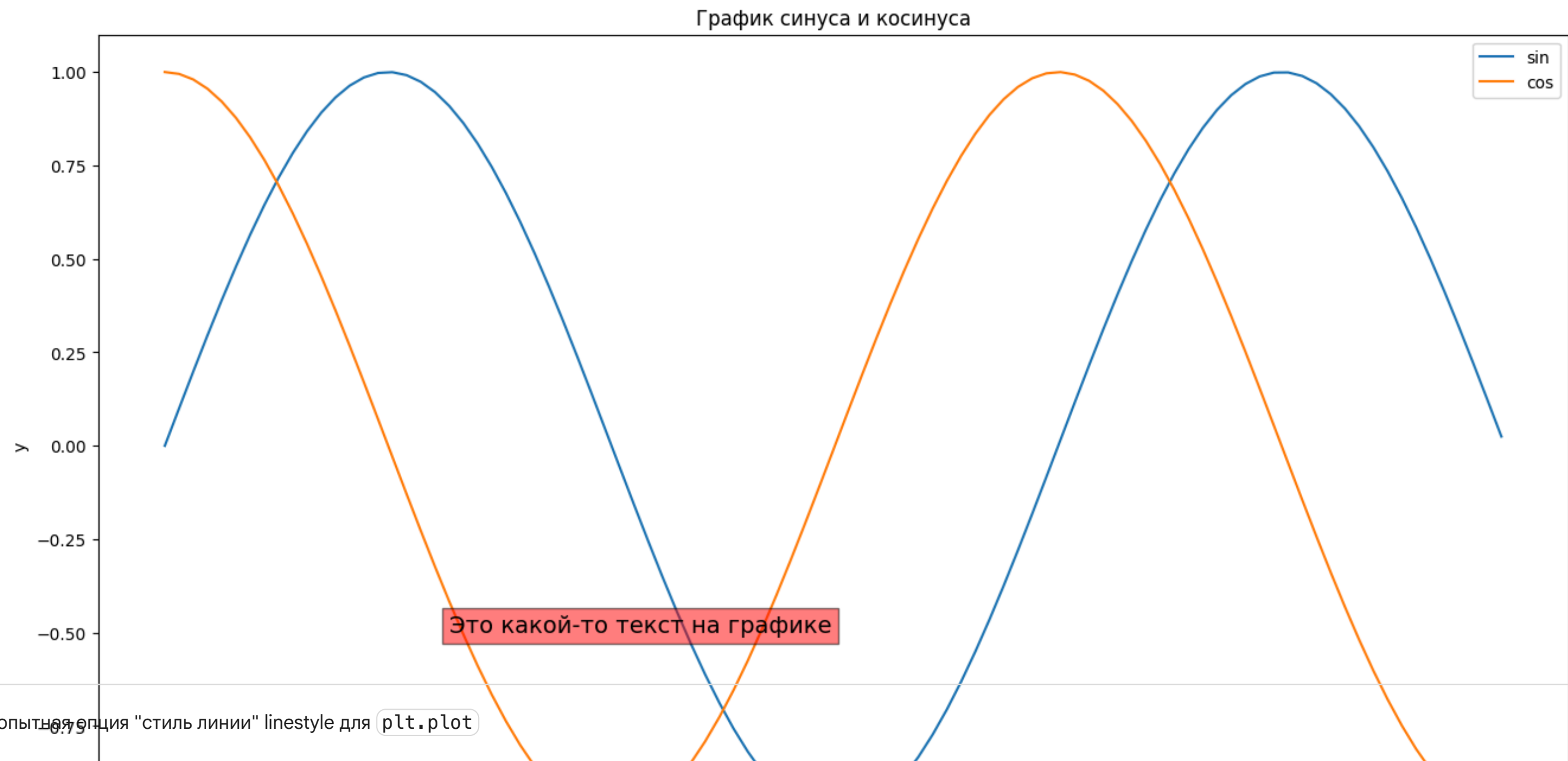
```
x = np.arange(0, 3 * np.pi, 0.1)
###
y_sin = np.sin(x)
y_cos = np.cos(x)
###

plt.figure(figsize=(16,9))
###
plt.plot(x, y_sin)
plt.plot(x, y_cos)
###
plt.title("График синуса и косинуса")
plt.xlabel("x")
plt.ylabel("y")
###
plt.legend(['sin', 'cos'])
###
plt.show()
```



```
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

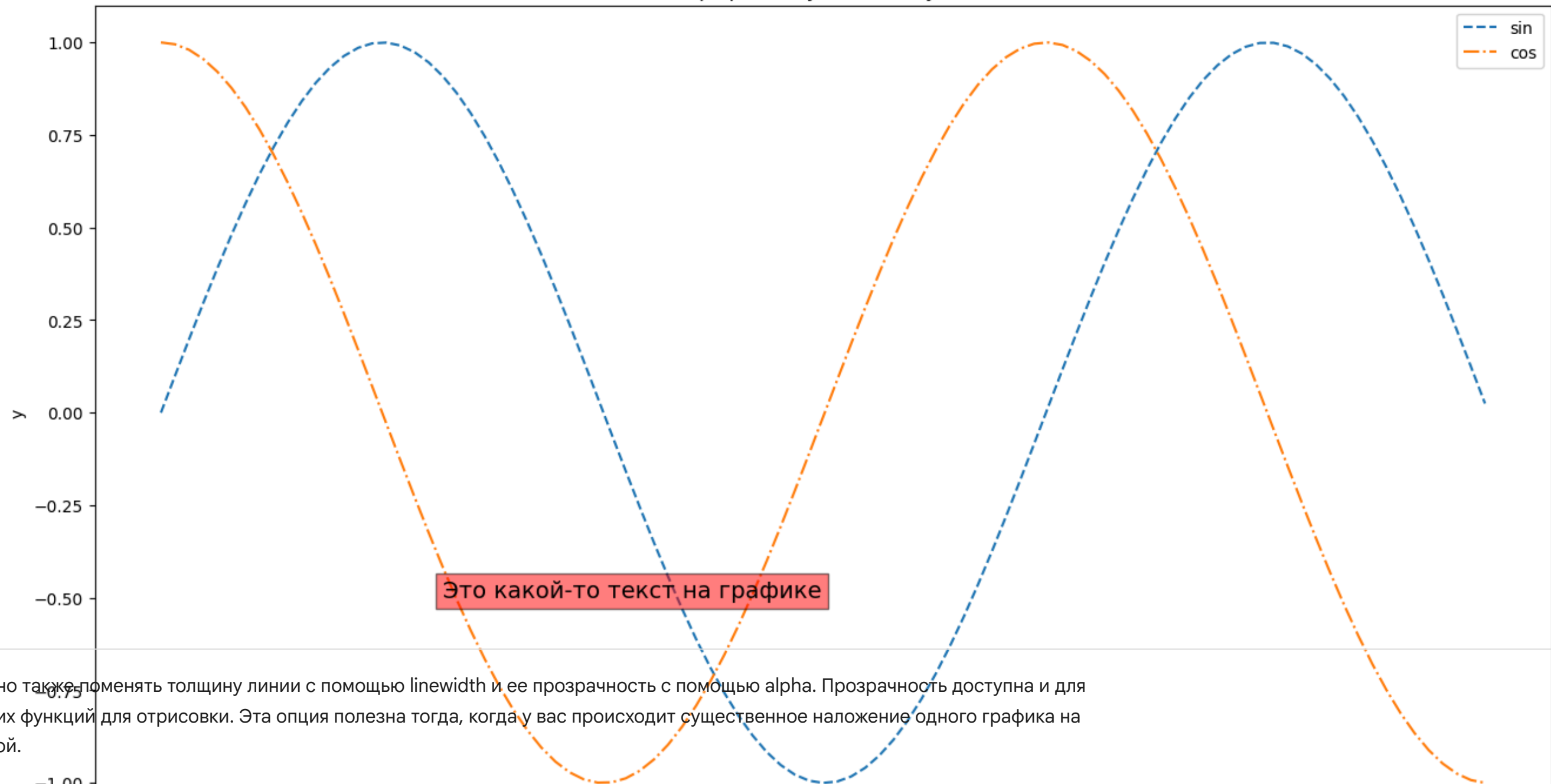
plt.figure(figsize=(16,9))
plt.plot(x, y_sin)
plt.plot(x, y_cos)
plt.title("График синуса и косинуса")
plt.xlabel("x")
plt.ylabel("y")
plt.legend(['sin', 'cos'])
###
plt.text(2, -0.5, 'Это какой-то текст на графике', bbox=dict(facecolor='red', alpha=0.5), fontsize=14)
###
plt.show()
```



```
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

plt.figure(figsize=(16,9))
###
plt.plot(x, y_sin, linestyle='--')
plt.plot(x, y_cos, linestyle='dashdot')
###
plt.title("График синуса и косинуса")
plt.xlabel("x")
plt.ylabel("y")
plt.legend(['sin', 'cos'])
plt.text(2, -0.5, 'Это какой-то текст на графике', bbox=dict(facecolor='red', alpha=0.5), fontsize=14)
plt.show()
```

График синуса и косинуса

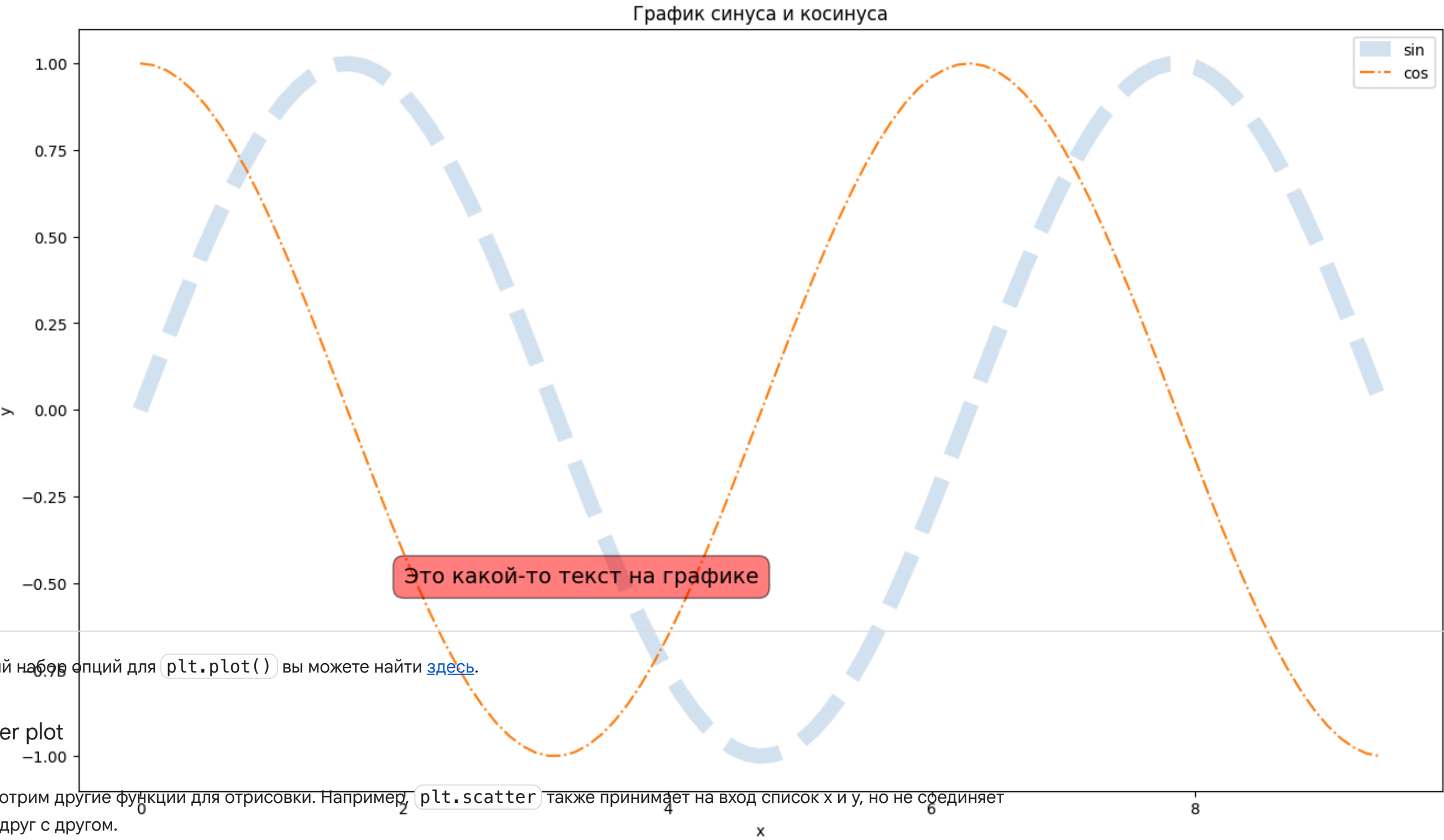


Можно также поменять толщину линии с помощью `linewidth` и ее прозрачность с помощью `alpha`. Прозрачность доступна и для других функций для отрисовки. Эта опция полезна тогда, когда у вас происходит существенное наложение одного графика на другой.

```
# Изменение толщины линии и прозрачности
x = np.arange(0, 3 * np.pi, 0.1)
y_sin = np.sin(x)
y_cos = np.cos(x)

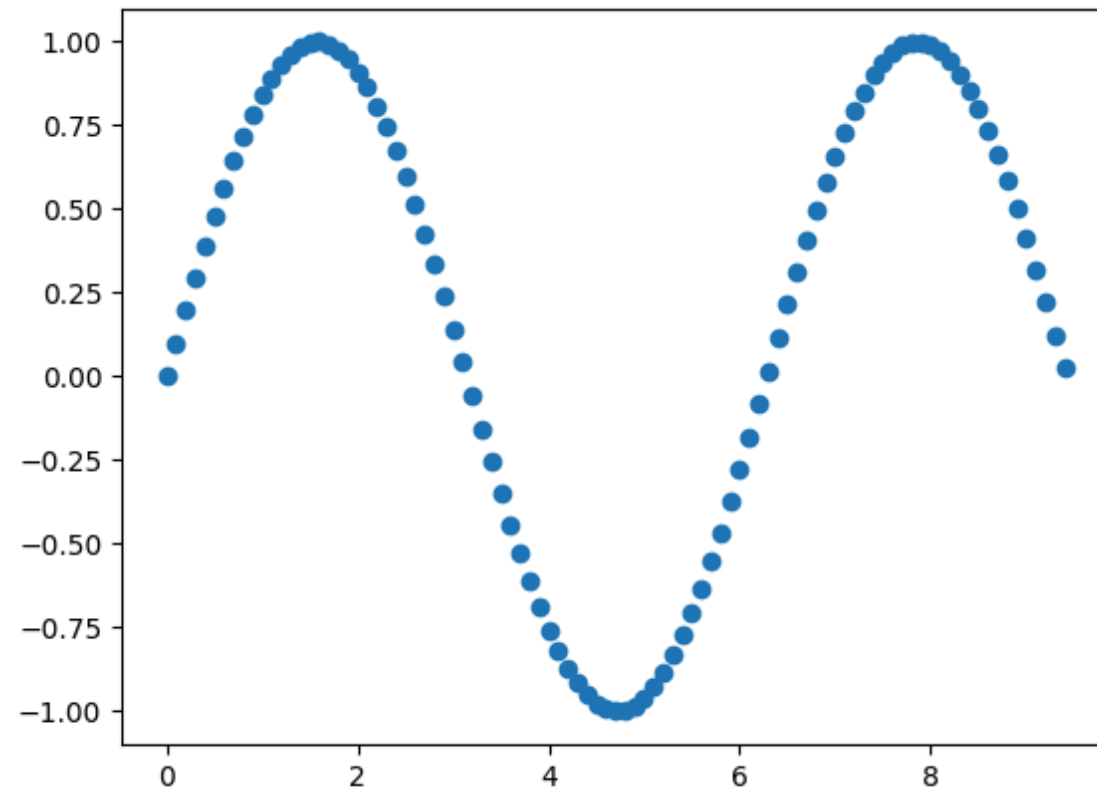
plt.figure(figsize=(16,9))
###
plt.plot(x, y_sin, linestyle='--', linewidth=10, alpha=0.2)
###
plt.plot(x, y_cos, linestyle='dashdot')
plt.title("График синуса и косинуса")
plt.xlabel("x")
plt.ylabel("y")
plt.legend(['sin', 'cos'])
###
plt.text(2, -0.5, 'Это какой-то текст на графике', bbox=dict(facecolor='red', alpha=0.5, boxstyle = 'round,pad=0.5'), fontsize=14)
```

```
###  
plt.show()
```



```
plt.scatter(x, y_sin)
```

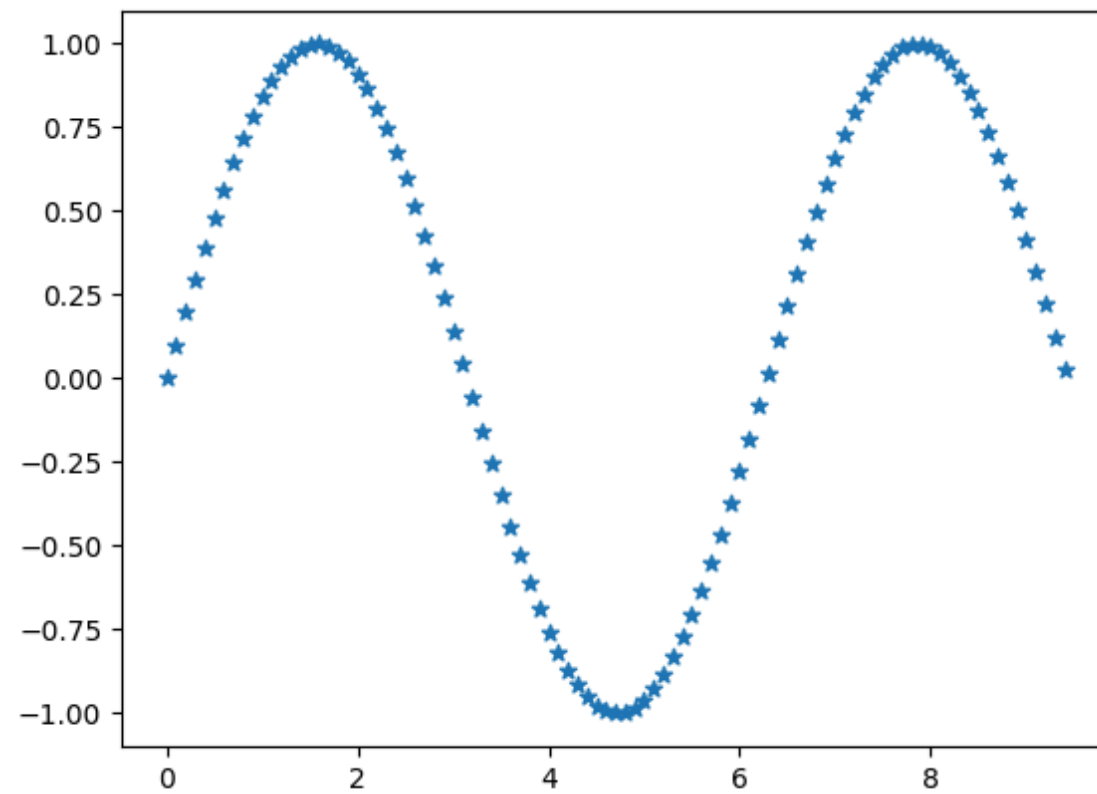
```
<matplotlib.collections.PathCollection at 0x7b6469d433d0>
```



Мы можем, например, менять форму точки

```
plt.scatter(x, y_sin, marker='*',)
```

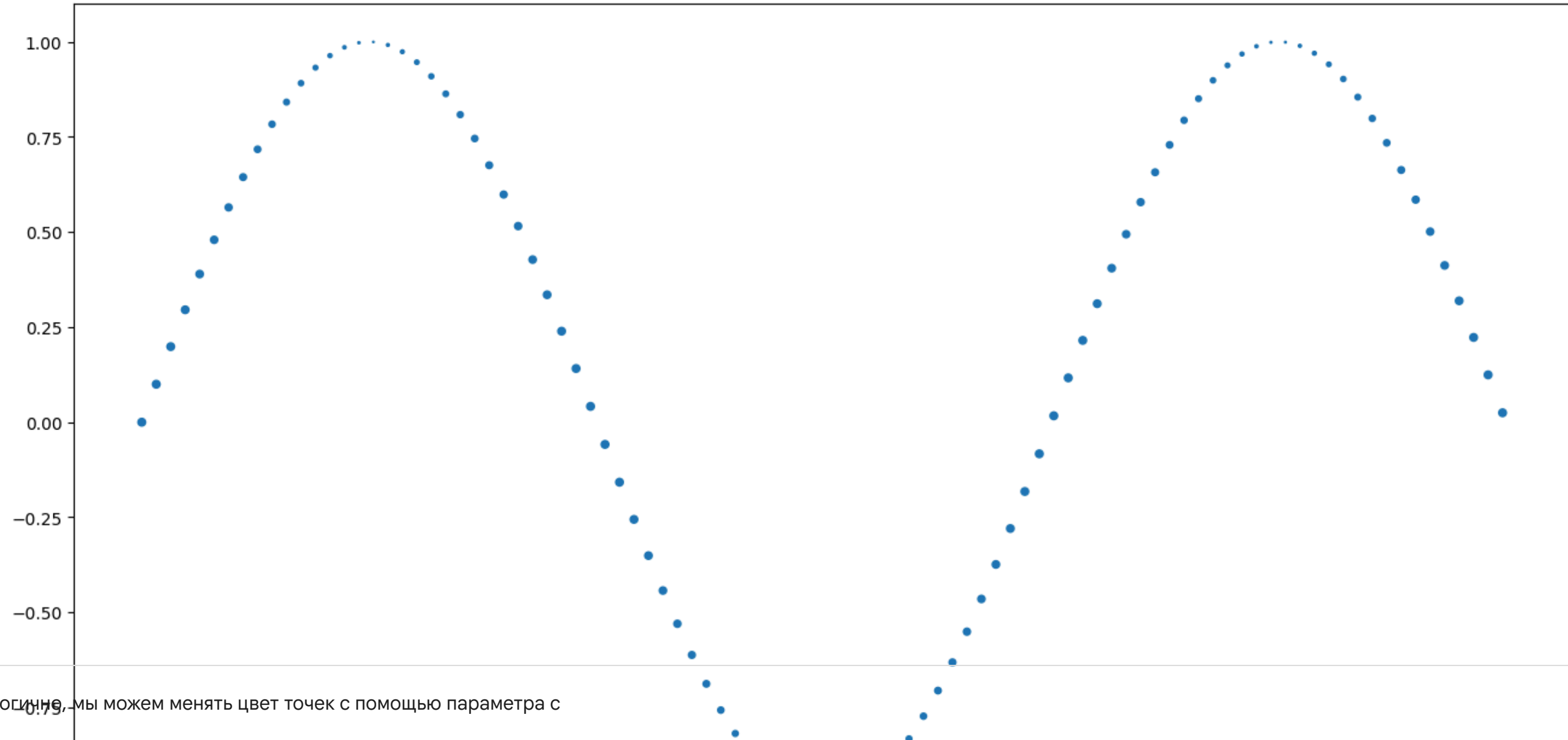
```
<matplotlib.collections.PathCollection at 0x7b64589d8d10>
```



Если мы передадим в опции `s` список такого же размера, что `x` и `y`, то у нас будет меняться размер точек в соответствии с тем, который мы укажем

```
plt.figure(figsize=(16,9))
plt.scatter(x, y_sin, s=np.abs(y_cos)*20)
```

<matplotlib.collections.PathCollection at 0x7b645871e850>



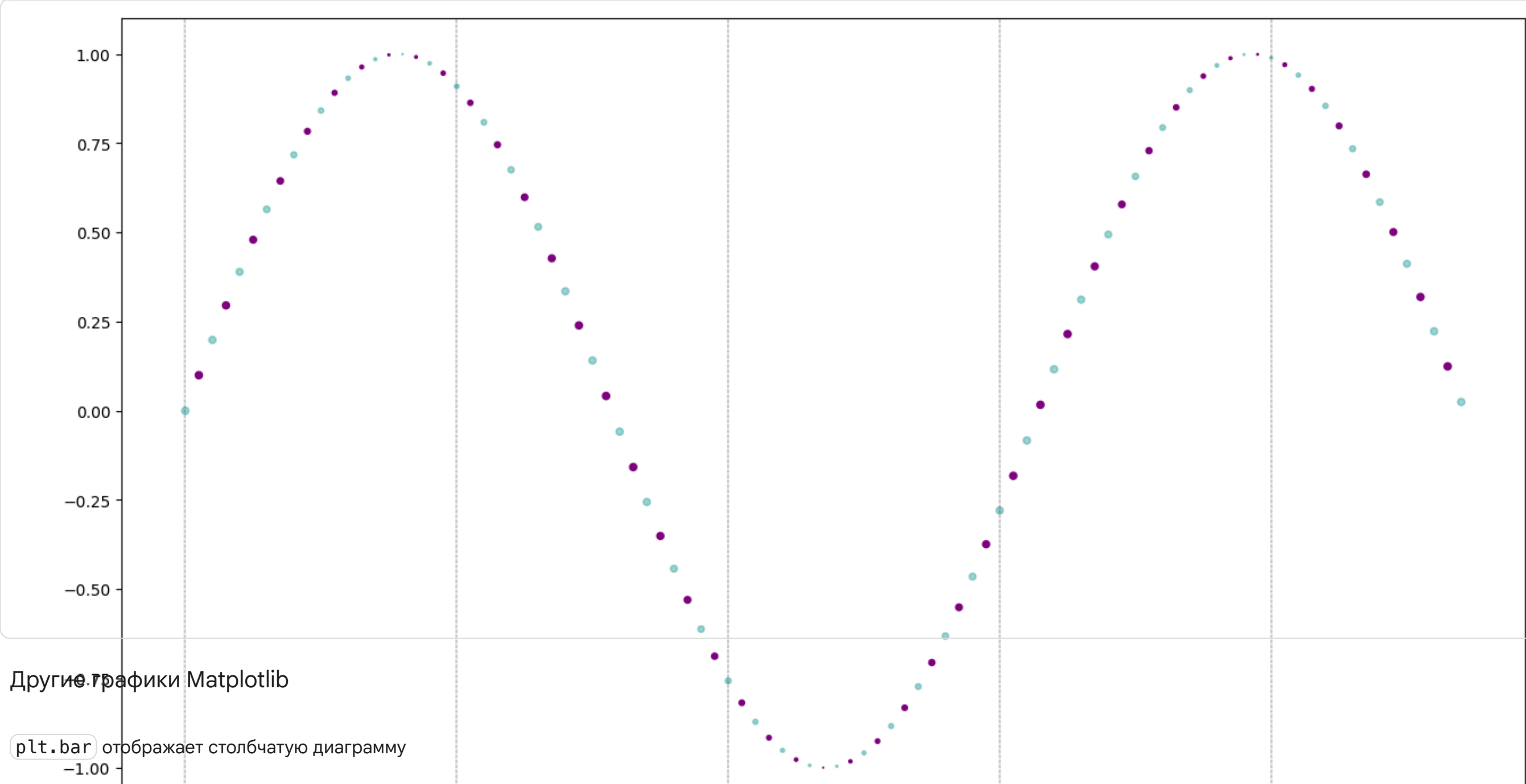
Аналогично, мы можем менять цвет точек с помощью параметра `c`

```
plt.figure(figsize=(16,9))
colors_glossary = ['#39AAA67F', 'purple']

# for i in range(x.shape[0]):
#     colors.append(colors_glossary[i % 2])

colors = [colors_glossary[i % 2] for i in range(x.shape[0])]
plt.scatter(x, y_sin, s=np.abs(y_cos)*20, c=colors)

plt.grid(axis = 'x', color='black', linestyle='-.', linewidth=0.3)
```

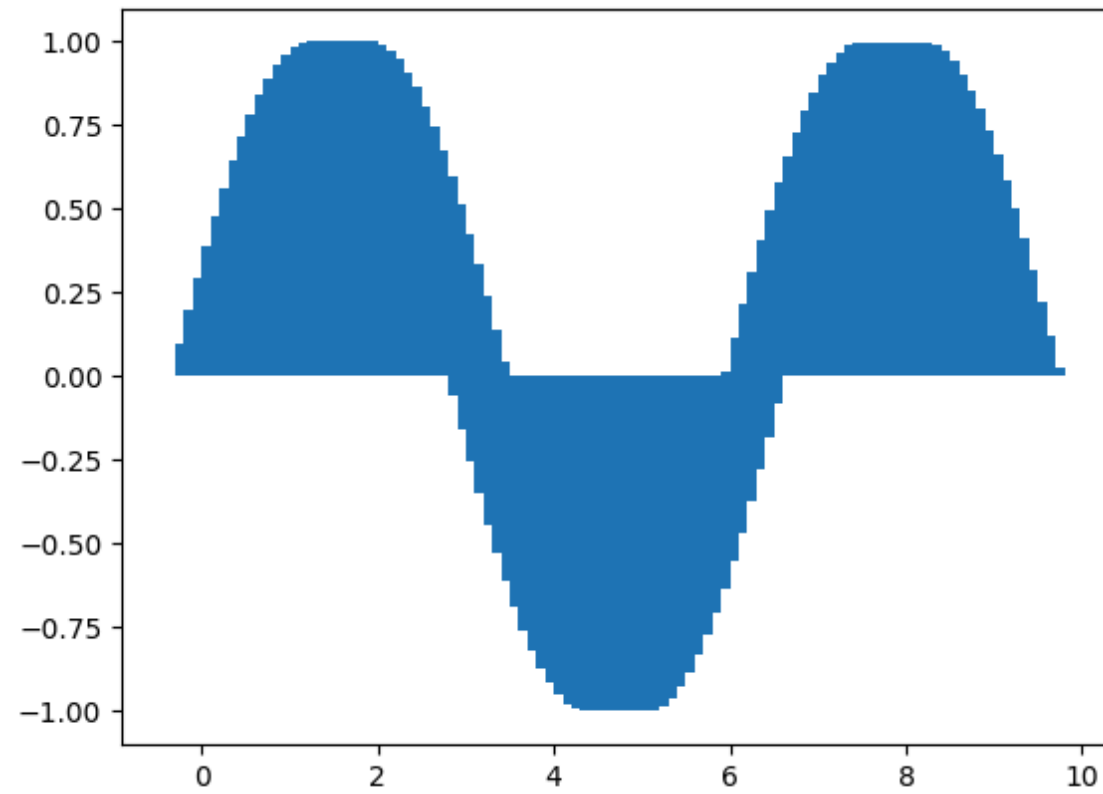


Другие графики Matplotlib

`plt.bar` отображает столбчатую диаграмму

```
plt.bar(x, y_sin)
```

<BarContainer object of 95 artists>



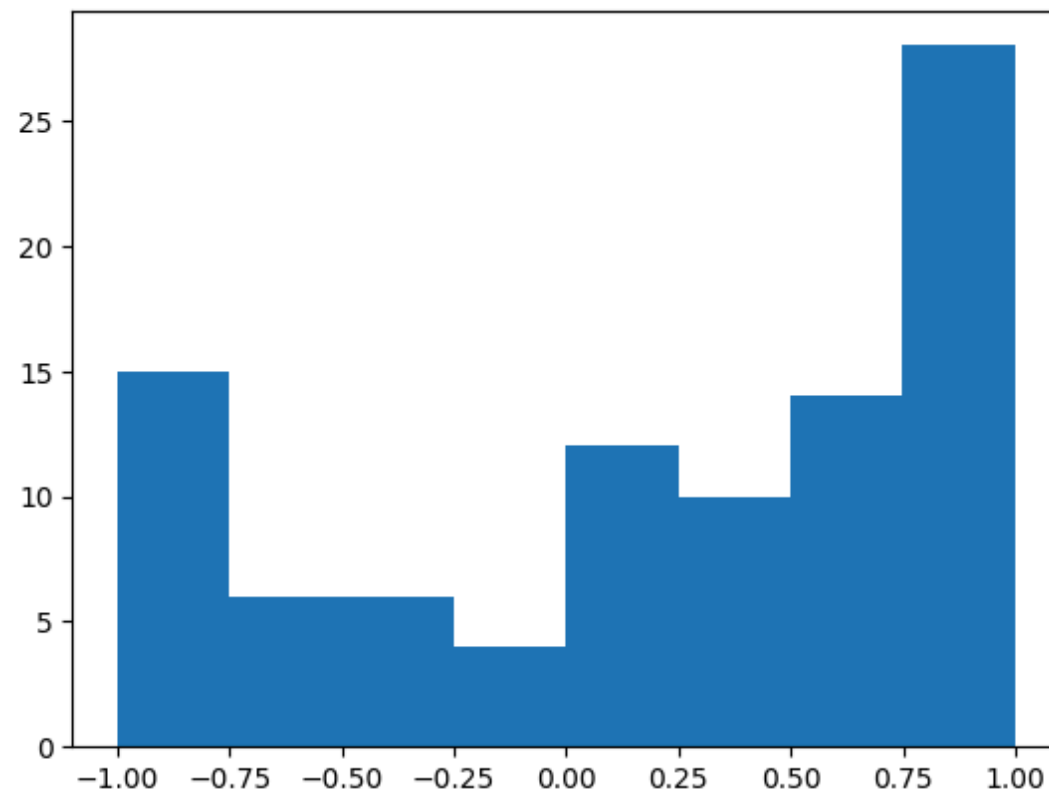
▼ Вопрос

Чем отличается barplot от гистограммы?

plt.hist принимает всего один список на вход, и отрисовывает частоту встречаемости значений в списке по интервалам (бакетам или бинам)

```
plt.hist(y_sin, bins='auto') # bins регулирует число столбиков
```

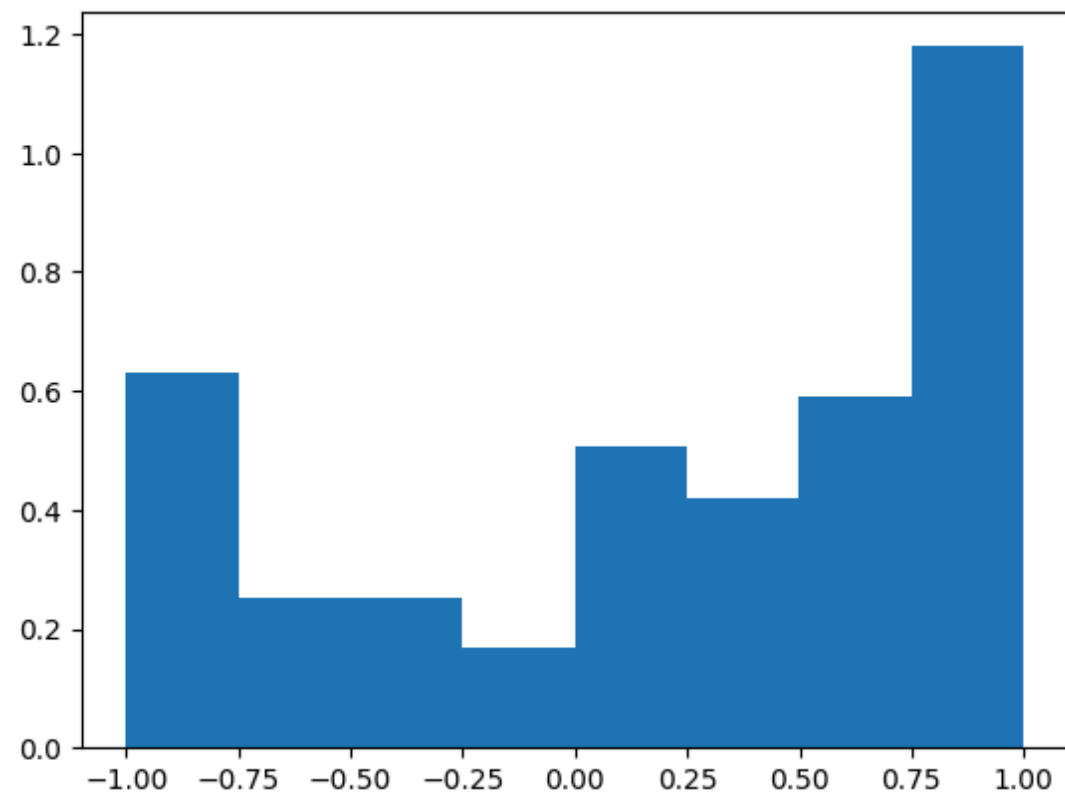
```
(array([15., 6., 6., 4., 12., 10., 14., 28.]),  
array([-9.99923258e-01, -7.49986150e-01, -5.00049042e-01, -2.50111935e-01,  
       -1.74827261e-04,  2.49762280e-01,  4.99699388e-01,  7.49636495e-01,  
        9.99573603e-01]),  
<BarContainer object of 8 artists>)
```



Если мы передадим параметр `density=True`, то получим нормированную гистограмму, то есть некоторого рода оценку плотности вероятности встречаемости значений нашего списка. Таким образом "площадь под графиком" будет равна 1.

```
plt.hist(y_sin, bins='auto', density=True)
```

```
(array([0.63173787, 0.25269515, 0.25269515, 0.16846343, 0.5053903 ,
        0.42115858, 0.58962202, 1.17924403]),
 array([-9.99923258e-01, -7.49986150e-01, -5.00049042e-01, -2.50111935e-01,
        -1.74827261e-04,  2.49762280e-01,  4.99699388e-01,  7.49636495e-01,
        9.99573603e-01]),
 <BarContainer object of 8 artists>)
```



Больше информации и всевозможных параметров вы можете найти в документации:

1. [plt.scatter](#)
2. [plt.bar](#)
3. [plt.hist](#)

▼ Функциональный и объектный подход в Matplotlib

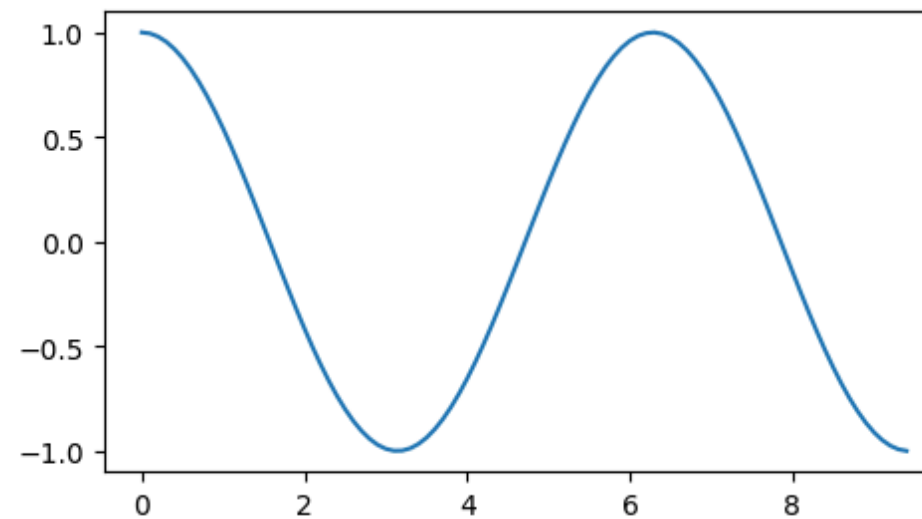
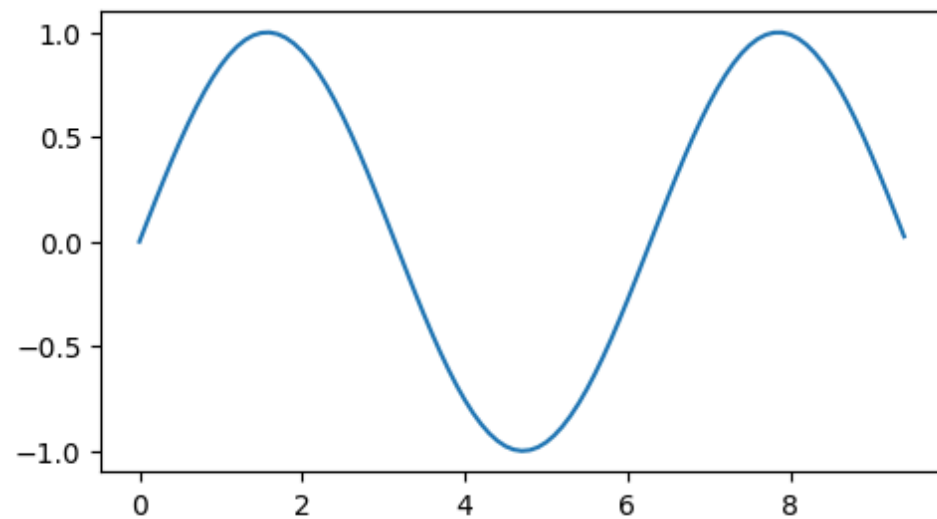
С помощью глобального объекта `plt` мы можем рисовать несколько пар осей на одном "полотне". Как уже упоминалось, это представляет собой так называемый первый или функциональный подход.

`plt.subplot` принимает первым параметром кол-во строк сетки, вторым кол-во столбцов сетки, а последний параметр устанавливает ту пару осей с которой мы работаем сейчас.

Рекомендую потренироваться с этим примером самостоятельно.

```
plt.figure(figsize=(12,3))
plt.subplot(1,2,1) # с этого момента рисуем на паре осей 1
#
#
plt.plot(x, y_sin)
#
```

```
#
plt.subplot(1,2,2) # с этого момента рисуем на паре осей 2
#
#
plt.plot(x, y_cos)
#
#
plt.show()
```

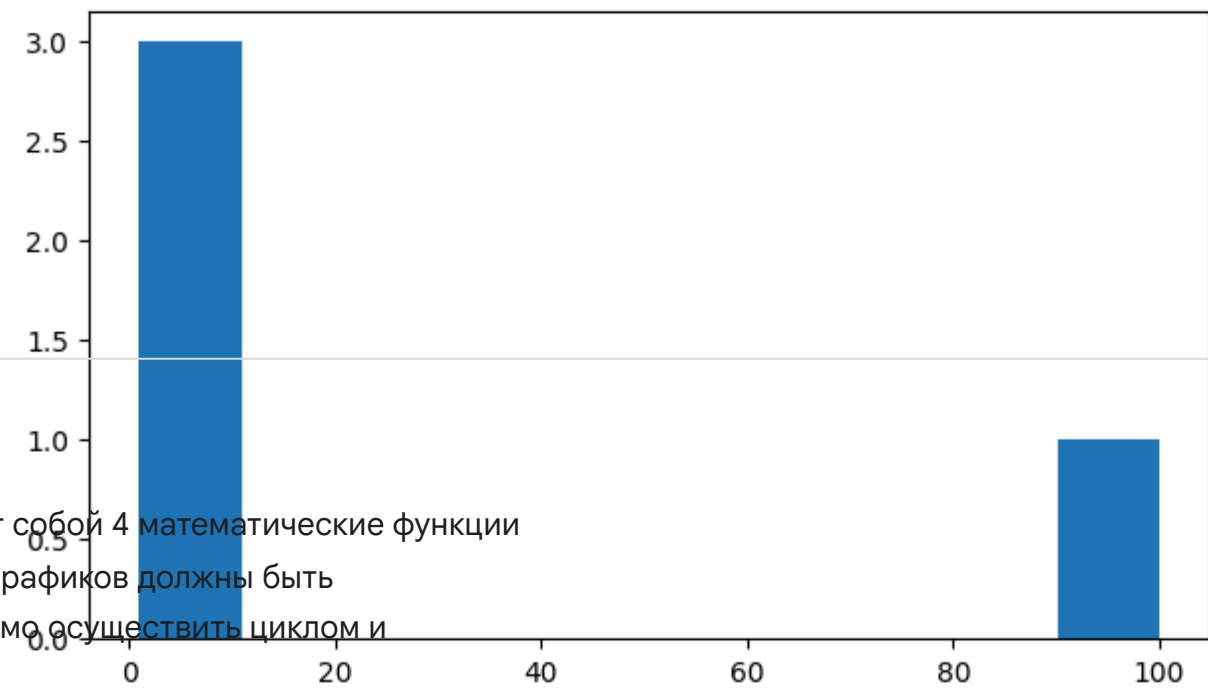
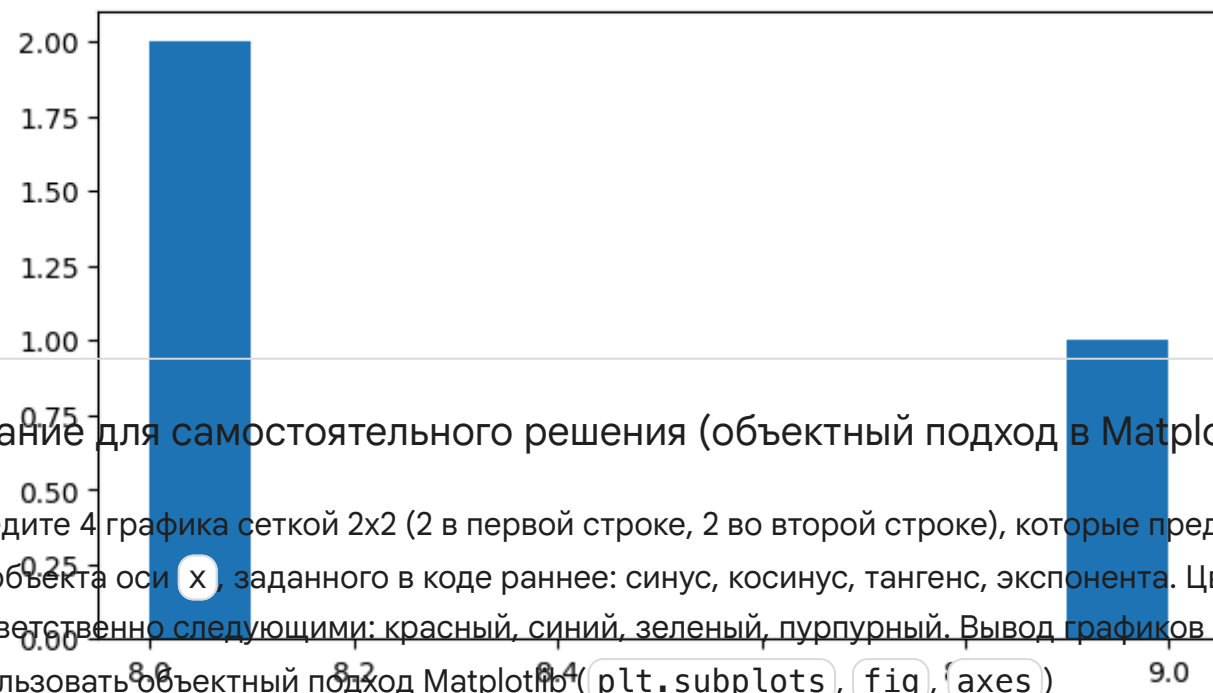
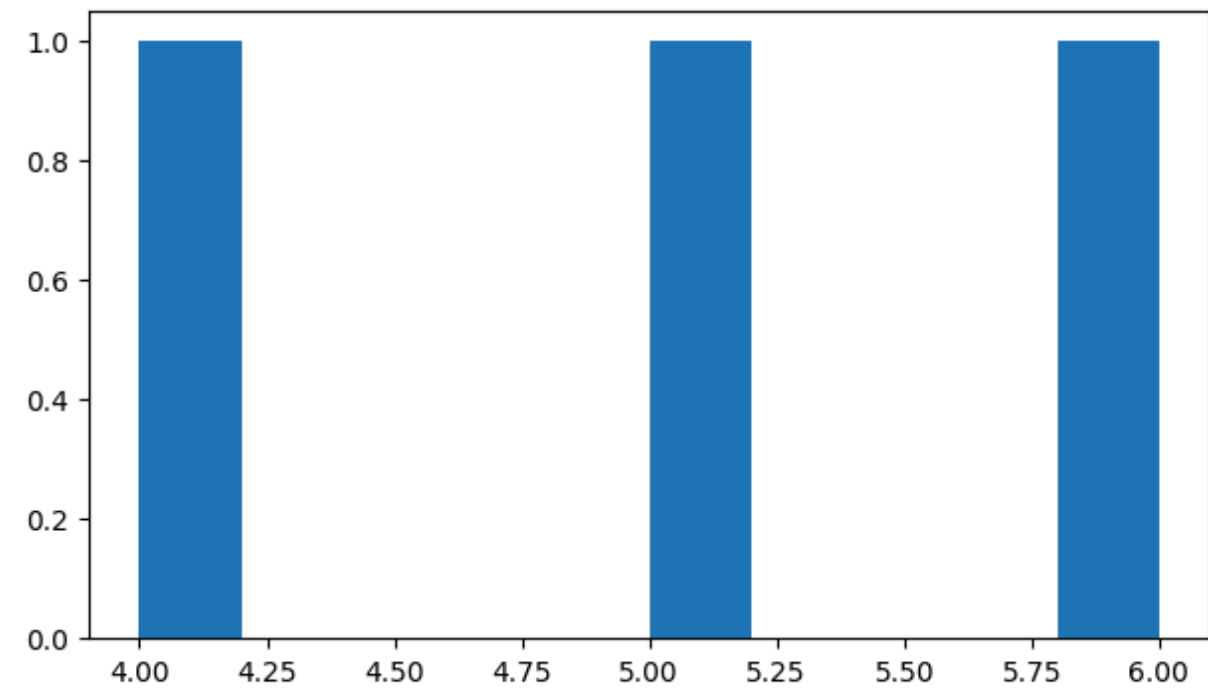
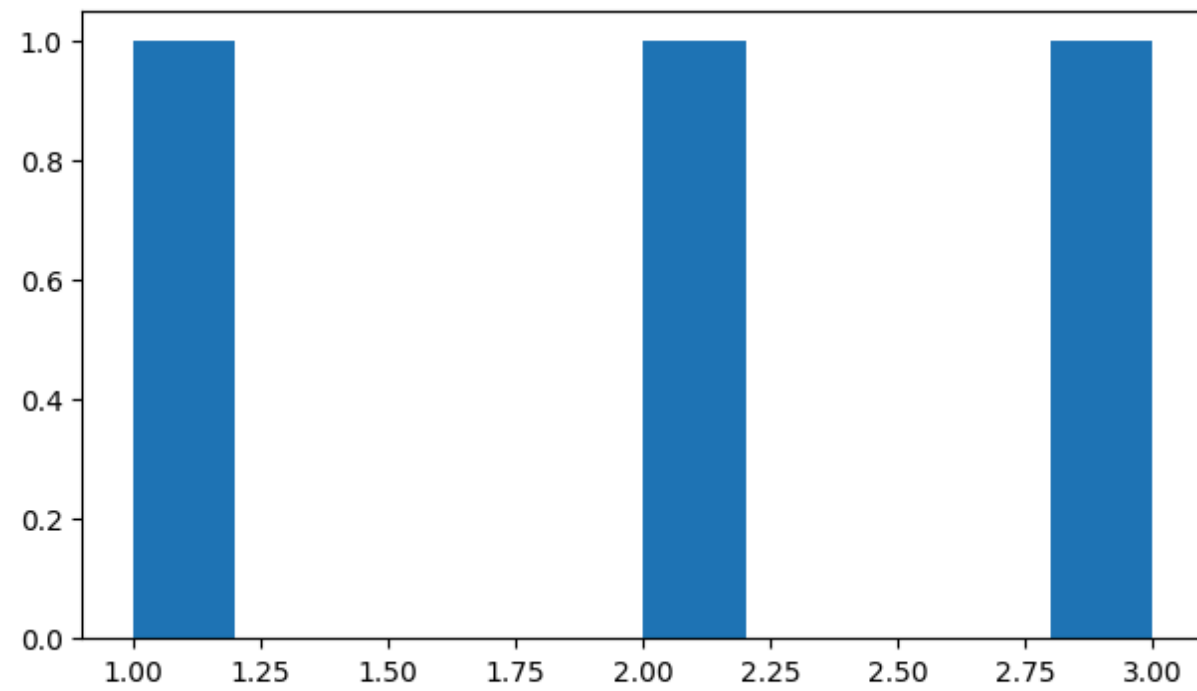


Ниже мы покажем альтернативный способ отображать несколько графиков на одном полотне. Здесь используется так называемый объектный интерфейс, который мы уже упоминали выше.

[plt.subplots\(\)](#) также принимает кол-во строк и столбцов, а также все те параметры, что принимает `plt.figure()`. Возвращает два объекта: объект фигуры и набор объектов осей в виде пипру массива размерности кол-во строк на кол-во столбцов.

```
fig, axes = plt.subplots(2,2, figsize=(16,9))

l = [[1,2,3], [4,5,6], [8,8,9], [100,1,2,3]]
for idx, ax in enumerate(axes.ravel()):
    ax.hist(l[idx])
```

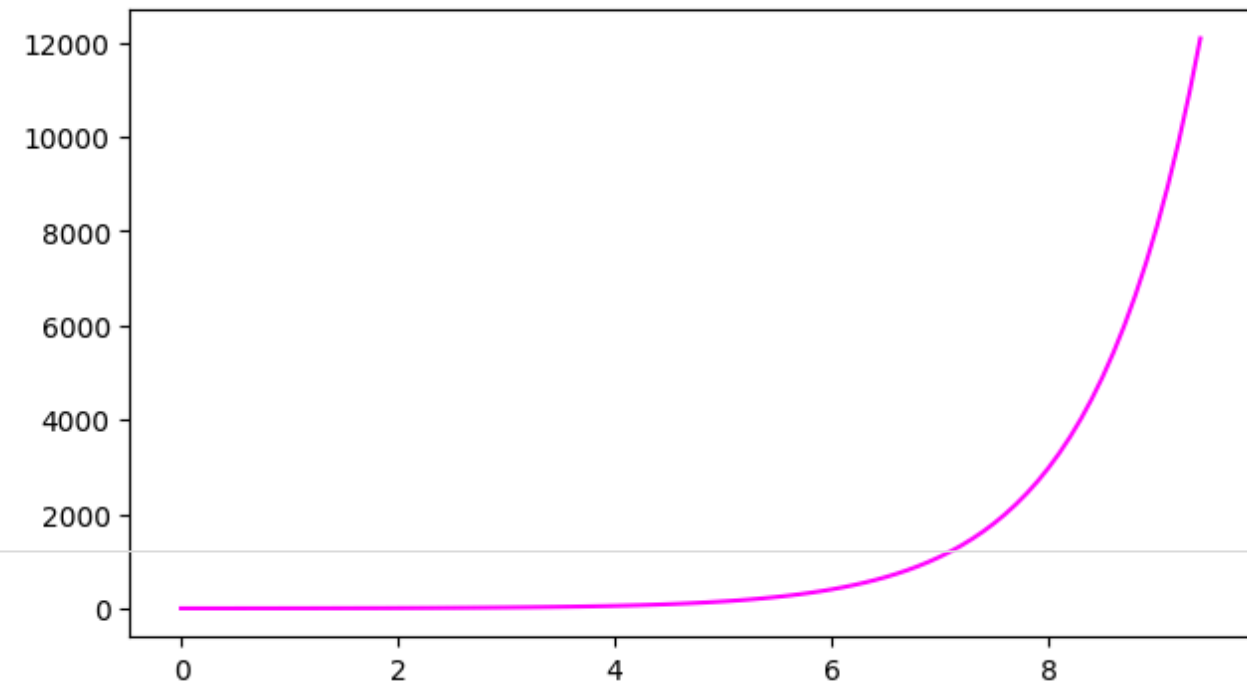
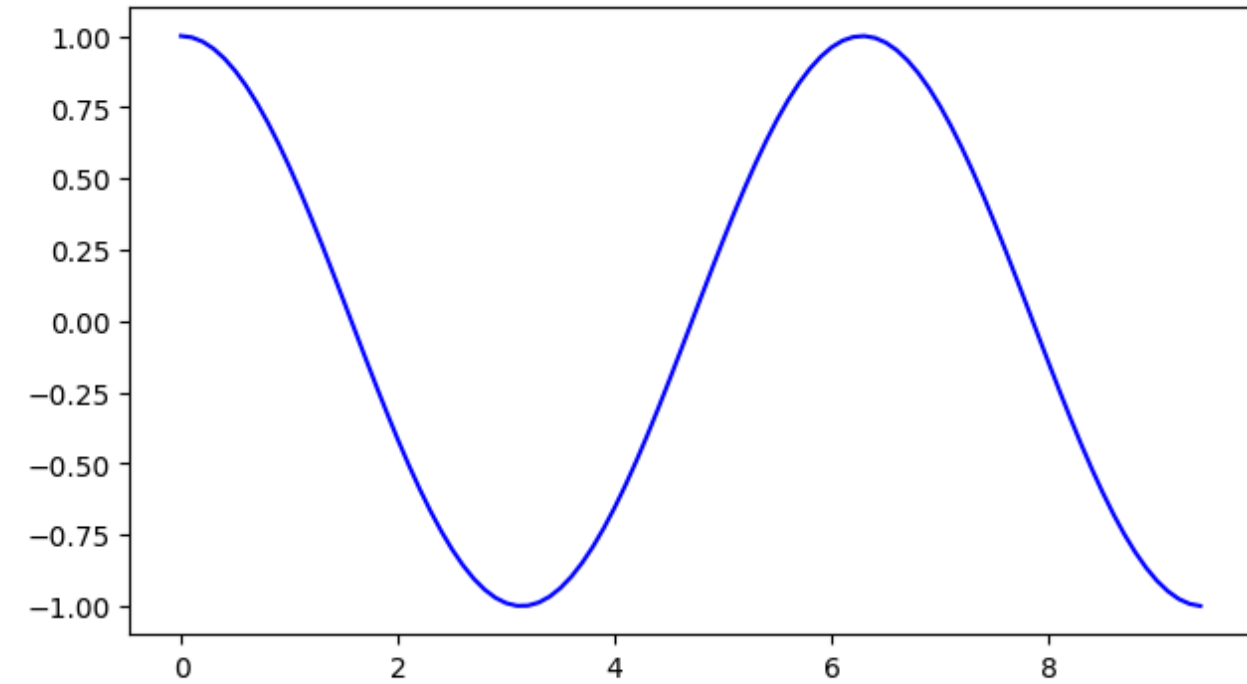
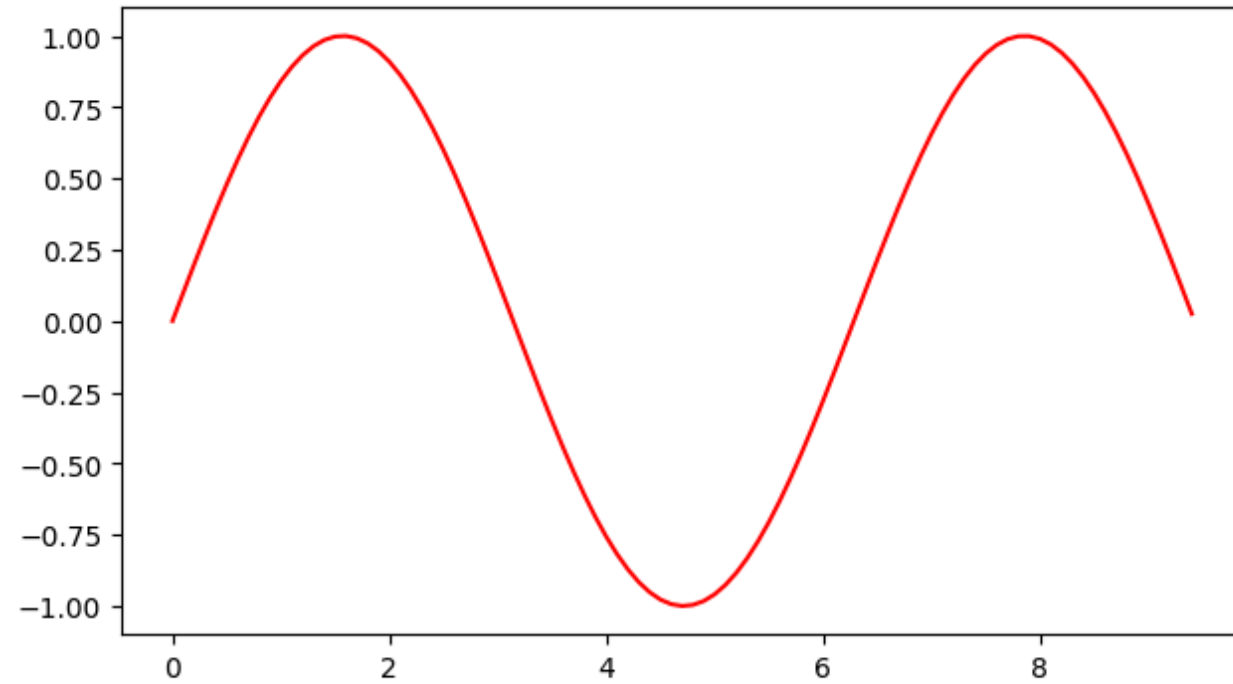


▼ Задание для самостоятельного решения (объектный подход в Matplotlib)

Выведите 4 графика сеткой 2x2 (2 в первой строке, 2 во второй строке), которые представляют собой 4 математические функции для объекта оси `x`, заданного в коде ранее: синус, косинус, тангенс, экспонента. Цвета для графиков должны быть соответственно следующими: красный, синий, зеленый, пурпурный. Вывод графиков необходимо осуществить циклом и использовать объектный подход Matplotlib (`plt.subplots`, `fig`, `axes`)

your code here

[<Axes: > <Axes: > <Axes: > <Axes: >]



Раздел 2. Matplotlib на реальных данных. Практика

Ну что же — пришло время Make visualizations great again :)

Поработаем с реальными датасетами!!

Ссылки на данные и различную справочную информацию для дополнительного изучения

https://drive.google.com/file/d/1eBnVjlbDuSRtSUhqrR_TEUiRaz_t7QDH/view?usp=sharing

<https://drive.google.com/file/d/1i3l2gC4US6fSlg4ett1ZttDLWwda0dcu/view?usp=sharing>

[What is money supply?](#)

The United States Money Supply M0 is the most liquid measure of the money supply including coins and notes in circulation and other assets that are easily convertible into cash. Money Supply M0 and M1, are also known as narrow money.

What is bitcoin?

```
from google.colab import files
uploaded = files.upload()
```

Выбрать файлы

Файл не выбран

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving historical_country_United_States_indicator_Money_Supply_M0.csv to historical_country_United_States_indicator_Money_Supply_M0.csv

Saving BTC-USD.csv to BTC-USD.csv

```
!ls
```

BTC-USD.csv historical_country_United_States_indicator_Money_Supply_M0.csv sample_data

```
import pandas as pd

btc_usd = pd.read_csv('BTC-USD.csv')
money_supply = pd.read_csv('historical_country_United_States_indicator_Money_Supply_M0.csv')
```

btc_usd

	Date	Open	High	Low	Close	Adj Close	Volume
0	2014-10-01	387.427002	411.697998	289.295990	338.321014	338.321014	902994450
1	2014-11-01	338.649994	457.092987	320.626007	378.046997	378.046997	659733360
2	2014-12-01	378.248993	384.037994	304.231995	320.192993	320.192993	553102310
3	2015-01-01	320.434998	320.434998	171.509995	217.464005	217.464005	1098811912
4	2015-02-01	216.867004	265.610992	212.014999	254.263000	254.263000	711518700
...	...	...	...	...	...	...	...
74	2020-12-01	19633.769531	29244.876953	17619.533203	29001.720703	29001.720703	1212259707946
75	2021-01-01	28994.009766	41946.738281	28722.755859	33114.359375	33114.359375	2155904438233
76	2021-02-01	33114.578125	58330.570313	32384.228516	45137.769531	45137.769531	2267152936675
77	2021-03-01	45159.503906	61683.863281	45115.093750	58870.894531	58870.894531	893379837811
78	2021-03-19	57699.785156	58873.882813	56660.320313	58573.390625	58573.390625	56108134400

79 rows × 7 columns

```
btc_usd.dtypes
```

0

Date	object
Open	float64
High	float64
Low	float64
Close	float64
Adj Close	float64
Volume	int64

dtype: object

```
btc_usd = btc_usd.astype({"Date": "datetime64[s]"})
btc_usd
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2014-10-01	387.427002	411.697998	289.295990	338.321014	338.321014	902994450
1	2014-11-01	338.649994	457.092987	320.626007	378.046997	378.046997	659733360
2	2014-12-01	378.248993	384.037994	304.231995	320.192993	320.192993	553102310
3	2015-01-01	320.434998	320.434998	171.509995	217.464005	217.464005	1098811912
4	2015-02-01	216.867004	265.610992	212.014999	254.263000	254.263000	711518700
...	...	...	...	...	...	...	...
74	2020-12-01	19633.769531	29244.876953	17619.533203	29001.720703	29001.720703	1212259707946
75	2021-01-01	28994.009766	41946.738281	28722.755859	33114.359375	33114.359375	2155904438233
76	2021-02-01	33114.578125	58330.570313	32384.228516	45137.769531	45137.769531	2267152936675
77	2021-03-01	45159.503906	61683.863281	45115.093750	58870.894531	58870.894531	893379837811
78	2021-03-19	57699.785156	58873.882813	56660.320313	58573.390625	58573.390625	56108134400

79 rows x 7 columns

btc_usd.dtypes

0	
Date	datetime64[s]
Open	float64
High	float64
Low	float64
Close	float64
Adj Close	float64
Volume	int64

dtype: object

money_supply.head()

	Country	Category	DateTime	Value	Frequency	HistoricalDataSymbol	LastUpdate
0	United States	Money Supply M0	1959-01-31T00:00:00	50463	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
1	United States	Money Supply M0	1959-02-28T00:00:00	49805	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
2	United States	Money Supply M0	1959-03-31T00:00:00	49733	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
3	United States	Money Supply M0	1959-04-30T00:00:00	50058	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
4	United States	Money Supply M0	1959-05-31T00:00:00	50112	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00

money_supply.dtypes

0	
Country	object
Category	object
DateTime	object
Value	int64
Frequency	object
HistoricalDataSymbol	object
LastUpdate	object

dtype: object

money_supply.astype({"DateTime": "datetime64[s]"})

	Country	Category	DateTime	Value	Frequency	HistoricalDataSymbol	LastUpdate
0	United States	Money Supply M0	1959-01-31	50463	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
1	United States	Money Supply M0	1959-02-28	49805	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
2	United States	Money Supply M0	1959-03-31	49733	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
3	United States	Money Supply M0	1959-04-30	50058	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
4	United States	Money Supply M0	1959-05-31	50112	Monthly	UNITEDSTAMONSUPM0	2015-04-25T21:12:00
...	...	...	...	...	...	...	...
740	United States	Money Supply M0	2020-09-30	4880400	Monthly	UNITEDSTAMONSUPM0	2020-10-17T10:11:00
741	United States	Money Supply M0	2020-10-31	4917100	Monthly	UNITEDSTAMONSUPM0	2020-11-14T10:08:00
742	United States	Money Supply M0	2020-11-30	5093100	Monthly	UNITEDSTAMONSUPM0	2020-12-12T10:07:00
743	United States	Money Supply M0	2020-12-31	5206600	Monthly	UNITEDSTAMONSUPM0	2021-01-16T10:11:00
744	United States	Money Supply M0	2021-01-31	5247900	Monthly	UNITEDSTAMONSUPM0	2021-02-13T10:07:00

745 rows x 7 columns

money_supply['DateTime'].to_numpy().astype('datetime64[D]')

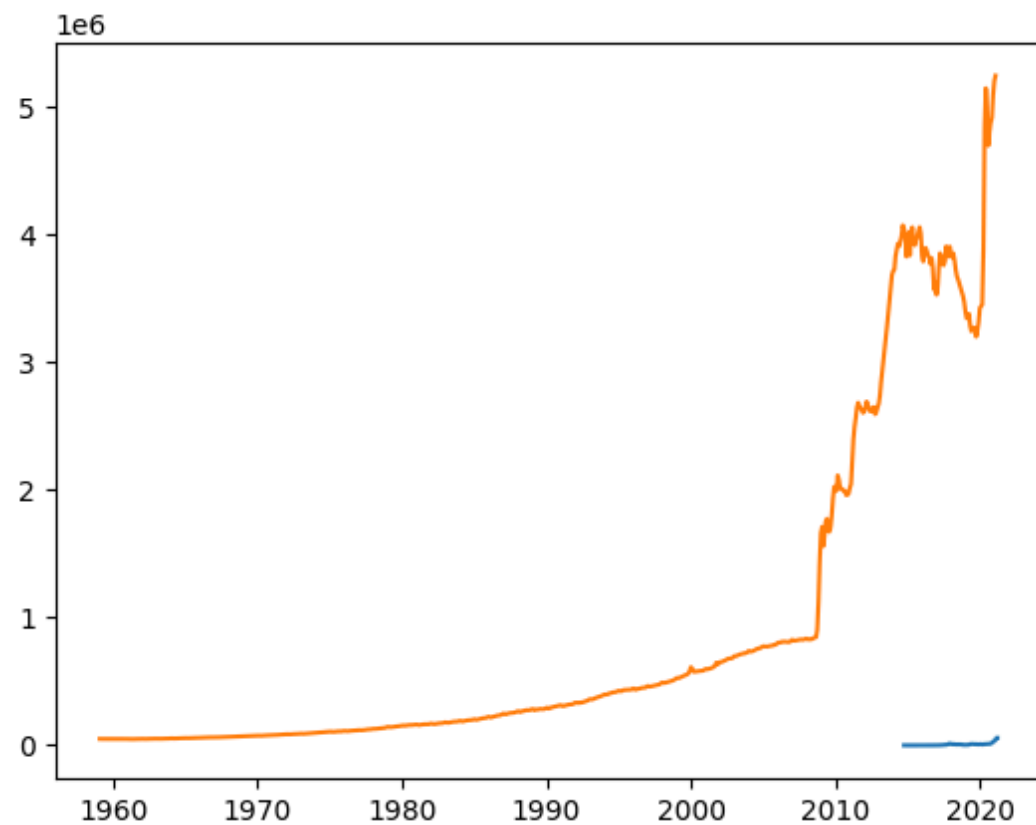
```
array(['1959-01-31', '1959-02-28', '1959-03-31', '1959-04-30',
      '1959-05-31', '1959-06-30', '1959-07-31', '1959-08-31',
      '1959-09-30', '1959-10-31', '1959-11-30', '1959-12-31',
      '1960-01-31', '1960-02-29', '1960-03-31', '1960-04-30',
      '1960-05-31', '1960-06-30', '1960-07-31', '1960-08-31',
      '1960-09-30', '1960-10-31', '1960-11-30', '1960-12-31',
      '1961-01-31', '1961-02-28', '1961-03-31', '1961-04-30',
      '1961-05-31', '1961-06-30', '1961-07-31', '1961-08-31',
      '1961-09-30', '1961-10-31', '1961-11-30', '1961-12-31',
      '1962-01-31', '1962-02-28', '1962-03-31', '1962-04-30',
      '1962-05-31', '1962-06-30', '1962-07-31', '1962-08-31',
      '1962-09-30', '1962-10-31', '1962-11-30', '1962-12-31',
      '1963-01-31', '1963-02-28', '1963-03-31', '1963-04-30',
      '1963-05-31', '1963-06-30', '1963-07-31', '1963-08-31',
      '1963-09-30', '1963-10-31', '1963-11-30', '1963-12-31',
      '1964-01-31', '1964-02-29', '1964-03-31', '1964-04-30',
      '1964-05-31', '1964-06-30', '1964-07-31', '1964-08-31',
      '1964-09-30', '1964-10-31', '1964-11-30', '1964-12-31',
      '1965-01-31', '1965-02-28', '1965-03-31', '1965-04-30',
      '1965-05-31', '1965-06-30', '1965-07-31', '1965-08-31',
      '1965-09-30', '1965-10-31', '1965-11-30', '1965-12-31',
      '1966-01-31', '1966-02-28', '1966-03-31', '1966-04-30',
      '1966-05-31', '1966-06-30', '1966-07-31', '1966-08-31',
      '1966-09-30', '1966-10-31', '1966-11-30', '1966-12-31',
      '1967-01-31', '1967-02-28', '1967-03-31', '1967-04-30',
      '1967-05-31', '1967-06-30', '1967-07-31', '1967-08-31',
      '1967-09-30', '1967-10-31', '1967-11-30', '1967-12-31',
      '1968-01-31', '1968-02-29', '1968-03-31', '1968-04-30',
      '1968-05-31', '1968-06-30', '1968-07-31', '1968-08-31',
      '1968-09-30', '1968-10-31', '1968-11-30', '1968-12-31',
      '1969-01-31', '1969-02-28', '1969-03-31', '1969-04-30',
      '1969-05-31', '1969-06-30', '1969-07-31', '1969-08-31',
```

```
'1969-09-30', '1969-10-31', '1969-11-30', '1969-12-31',
'1970-01-31', '1970-02-28', '1970-03-31', '1970-04-30',
'1970-05-31', '1970-06-30', '1970-07-31', '1970-08-31',
'1970-09-30', '1970-10-31', '1970-11-30', '1970-12-31',
'1971-01-31', '1971-02-28', '1971-03-31', '1971-04-30',
'1971-05-31', '1971-06-30', '1971-07-31', '1971-08-31',
'1971-09-30', '1971-10-31', '1971-11-30', '1971-12-31',
'1972-01-31', '1972-02-29', '1972-03-31', '1972-04-30',
'1972-05-31', '1972-06-30', '1972-07-31', '1972-08-31',
'1972-09-30', '1972-10-31', '1972-11-30', '1972-12-31',
'1973-01-31', '1973-02-28', '1973-03-31', '1973-04-30',
'1973-05-31', '1973-06-30', '1973-07-31', '1973-08-31',
'1973-09-30', '1973-10-31', '1973-11-30', '1973-12-31',
'1974-01-31', '1974-02-28', '1974-03-31', '1974-04-30',
'1974-05-31', '1974-06-30', '1974-07-31', '1974-08-31',
'1974-09-30', '1974-10-31', '1974-11-30', '1974-12-31',
'1975-01-31', '1975-02-28', '1975-03-31', '1975-04-30',
'1975-05-31', '1975-06-30', '1975-07-31', '1975-08-31',
'1975-09-30', '1975-10-31', '1975-11-30', '1975-12-31',
'1976-01-31', '1976-02-29', '1976-03-31', '1976-04-30',
'1976-05-31', '1976-06-30', '1976-07-31', '1976-08-31',
'1976-09-30', '1976-10-31', '1976-11-30', '1976-12-31',
'1977-01-31', '1977-02-28', '1977-03-31', '1977-04-30',
'1977-05-31', '1977-06-30', '1977-07-31', '1977-08-31',
'1977-09-30', '1977-10-31', '1977-11-30', '1977-12-31',
'1978-01-31', '1978-02-28', '1978-03-31', '1978-04-30',
```

```
money_supply_date = money_supply['DateTime'].to_numpy().astype('datetime64[D]')
money_supply_value = money_supply['Value'].to_numpy()
btc_usd_date = btc_usd['Date'].to_numpy().astype('datetime64[D]')
btc_usd_close = btc_usd['Close'].to_numpy()

plt.plot(btc_usd_date, btc_usd_close)
plt.plot(money_supply_date, money_supply_value)

plt.show()
```



Что скажете про такой график? :)

Красиво? :) Репрезентативно? :)

✓ Улучшение и оптимизация графиков

Итак, представляем вашему вниманию 10 в меру абстрактных шагов, которые помогут значительно улучшить ваши визуализации и проапгрейдить ваш скилл в Matplotlib

1. Вносите ясность — что вообще изображено?
2. Сравнивайте — располагаем связанную информацию рядом
3. Упрощайте — действительно ли нужны все точки, чтобы передать сообщение?
4. Фокусируйте внимание — нужен ли весь диапазон значений, имеет ли это смысл?
5. Поощряйте вопросы — можно выделить интересные ответы на возникающие вопросы прямо на графике
6. Пытайтесь ответить на вопрос “почему” — из графика должна в идеале быть понятно мысль без каких-либо комментариев
7. Оставляйте возможность валидировать визуализацию — можно оставить у нескольких точек подписи значений
8. Делайте разные визуализации — нет универсального типа визуализации, ищите то, что подходит лучше всего
9. Здоровый скептицизм — приводите референсы, ссылки. Спрашивайте других людей понятно или им что-то из графика
10. Детализируйте — самые интересные моменты, возможно, стоит рассмотреть отдельно

Ну что же, будем исправлять наш график по порядку, по шагам. Будем отмечать плюсиком, что добавили/исправили, чтобы нагляднее отслеживать все изменения.

```
# 1
plt.figure(figsize=(16,9)) # +

plt.plot(btc_usd_date, btc_usd_close)
plt.plot(money_supply_date, money_supply_value)

plt.title("Валютные интервенции и стоимость биткоина USD") # +

plt.show()
```



```
# 1-2
# Для того, чтобы использовать парные оси y, перейдем к объектной форме
fig, ax1 = plt.subplots(figsize=(16,9)) # +

color = 'tab:blue' # +
ax1.set_xlabel('Время') # +
ax1.set_ylabel('BTC/USD цена, $', color = color) # +
ax1.plot(btc_usd_date, btc_usd_close, color = color) # +
ax1.tick_params(axis='y', labelcolor = color) # +

# Правая ось y будет для валютных интервенций
ax2 = ax1.twinx() # +
```

```

color = 'tab:purple' # +
ax2.set_ylabel('Объем валютных интервенций, млн $', color = color) # +
ax2.plot(money_supply_date, money_supply_value, color = color) # +
ax2.tick_params(axis = 'y', labelcolor = color) # +

plt.title("Валютные интервенции и стоимость биткоина USD")

plt.show()

```



```

# 3-4
fig, ax1 = plt.subplots(figsize=(16,9))

color = 'tab:blue'
ax1.set_xlabel('Время')
max_date = np.max(np.concatenate([btc_usd_date, money_supply_date])) # +
ax1.set_xlim(left=np.datetime64('2014-01-01'), right=max_date) # +
ax1.set_ylabel('BTC/USD цена, $', color = color)
ax1.plot(btc_usd_date, btc_usd_close, color = color)
ax1.tick_params(axis = 'y', labelcolor = color)

ax2 = ax1.twinx()

color = 'tab:purple'
ax2.set_ylabel('Объем валютных интервенций, млн $', color = color)

```

```
ax2.plot(money_supply_date, money_supply_value, color = color)
ax2.tick_params(axis = 'y', labelcolor = color)
```

```
plt.title("Валютные интервенции и стоимость биткоина USD")
```

